

1. Write a C# program to demonstrate five types of constructor in C#.

◆ 1 Default Constructor

- A constructor with **no parameters**.
- Provided automatically if you don't define any constructor.
- Initializes fields with default values.

◆ 2 Parameterized Constructor

- Takes arguments to initialize fields with specific values at the time of object creation.

◆ 3 Copy Constructor

- Takes another object of the same class as a parameter.
- Copies data from the passed object into the new object.

◆ 4 Static Constructor

- Declared with `static`.
- Called automatically **once per type**, before any object is created or static members are accessed.
- Used to initialize static fields.

◆ 5 Private Constructor

- Declared as `private`.
- Prevents instantiation from outside the class.
- Used in singleton patterns or static classes.



Summary Table

<i>Constructor Type</i>	<i>Purpose</i>
<i>Default</i>	Initializes fields with defaults
<i>Parameterized</i>	Initialize with specific values
<i>Copy</i>	Create a copy of another object
<i>Static</i>	Initialize static members, runs once
<i>Private</i>	Prevent external instantiation

```

using System;
// Constructor demo
namespace Sushanka_NCC_Lab
{
    internal class Person
    {
        string name;
        int age;
        private Person()    //1.Default Constructor / Parameter less constructor
        {
            name = ""; age = 0;
            Console.WriteLine($"I'm default constructor(no parameter).");
        }
        //2. Parameterized Constructor
        Person(string p_name)
        {
            name = p_name;
            Console.WriteLine($"I Parameterized Constructor. name: {name}");
        }
        //3. Copy Constructor
        Person(Person obj1)
        {
            name = ""; age = 0;
            Console.WriteLine($"I Copy Constructor. name: {obj1.name} ");
        }
        //4. Static Constructor
        static Person()
        {
            Console.WriteLine("-----Constructor Types-----");
            Console.WriteLine($"I' Static Constructor.");
        }
        //5. Private Constructor
        private Person(int x)
        {
            name = ""; age = x;
            Console.WriteLine($"I' Private Constructor. age: {age}");
        }

        static void Main()
        {
            Person p1 = new();
            Person p2 = new("Sushanka");
            Person p3 = new(p2);
            Person p5 = new(15);
        }
    }
}

```

```

-----Constructor Types-----
I' Static Constructor.
I'm default constructor(no parameter).
I' Parameterized Constructor. name: Sushanka
I' Copy Constructor. name: Sushanka
I' Private Constructor. age: 15

```

2. WAP in C# to demonstrate concept of auto Property and Read-Only Property.

◆ Auto Property

Meaning:

An automatically-implemented property that saves you from writing a separate field and boilerplate getter/setter.

C# automatically generates the backing field internally.

Why use it?

- Makes your code shorter and cleaner.
 - Useful for simple properties where no extra logic is needed in `get` or `set`.
-

◆ Read-Only Property

Meaning:

A property that only has a **getter**, so its value can only be set during object construction or initialization — and not changed afterward from outside the class.

Why use it?

- To make the property immutable (cannot be modified later).
 - Useful for properties that should be assigned once and remain constant for the lifetime of the object.
-

So:

Feature	Why use?
Auto Property	Simplicity & cleaner code for standard <code>get/set</code> .
Read-Only Property	To protect the value from being changed after initialization.

```

using System;

namespace Sushanka_NCC_Lab
{
    class Employee
    {
        // auto implement property: get and set
        string Name { get; set; }
        // Read-Only property: can only get, value set in constructor
        int Emp_Id { get; }
        // constructor
        Employee(string name, int id)
        {
            Name= name; // setting auto-implement property
            Emp_Id= id; // setting readonly property
        }

        static void Main()
        {
            Employee e1 = new("ram", 43);
            Console.WriteLine($"Employee's Name: {e1.Name} Id: {e1.Emp_Id}");

            // Modify auto-implemented property
            e1.Name = "Ram Bahadur";
            Console.WriteLine($"After update: \nEmployee's Name: {e1.Name} Id:
{e1.Emp_Id}");

            // modify read-only property
            //e1.Emp_Id = 1; // error it's readonly
        }
    }
}

```

```

Auto property & Read-Only property
Employee's Name: ram Id: 43
After update:
Employee's Name: Ram Bahadur Id: 43

```

3. WAP in C# to demonstrate jagged array.

A **jagged array** is an *array of arrays*.

Each element of the main array is itself an array, and **these inner arrays can have different lengths** — that's why it's called *jagged*.

🎯 Saves memory compared to a rectangular (2D) array if rows are uneven.

- Declared as an array of arrays (`int[] []`).

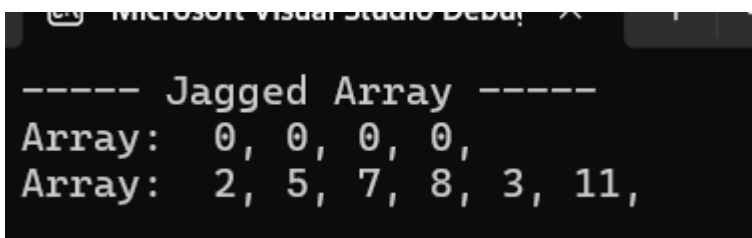
```
using System;

namespace Sushanka_NCC_Lab
{
    class Jagged
    {
        static void Main()
        {
            Console.WriteLine("----- Jagged Array -----");

            //int[][] jagged_arr = new int[2][];
            //jagged_arr[0] = new int[4] { 0, 0, 0, 0 };
            //jagged_arr[1] = new int[] { 2, 5, 7, 8, 3, 11 };

            //int[][] jagged_arr = [new int[] { 0, 0, 0, 0 }, new int[] { 2, 5, 7, 8, 3, 11 }];
            // simplified array declaration
            int[][] jagged_arr = [[ 0, 0, 0, 0 ], [2, 5, 7, 8, 3, 11]];

            foreach (int[] arr in jagged_arr)
            {
                Console.Write($"Array:\t");
                foreach (int i in arr)
                {
                    Console.Write($" {i}, ");
                }
                Console.WriteLine();
            }
        }
    }
}
```



```
Microsoft Visual Studio Debug Console
----- Jagged Array -----
Array:  0, 0, 0, 0,
Array:  2, 5, 7, 8, 3, 11,
```

4. WAP to demonstrate Indexer in C#: a) When index is of int type b) When index is of other than int type
- Indexers are members of a class or struct that allow instances of a class or struct to be indexed just like arrays

- Defined using the `this` keyword.
- Can have `get` and/or `set` accessors.

```
using System;
namespace Sushanka_NCC_Lab
{
    // primary constructor: allows to declare the constructor parameters directly in the
    // class declaration, instead of writing a separate constructor inside the body.
    class TestEmployee(int eid, string name, double salary)
    {
        // private fields
        int eid = eid;
        string name = name;
        double salary = salary;

        // indexer
        // [access modifier] [return type] [this keyword] [parameter list]
        public Object this [int index]
        {
            get
            {
                if (index == 0)
                    return eid;
                else if (index == 1)
                    return name;
                else if (index == 2)
                    return salary;
                return null;
            }
            set
            {
                if (index == 0)
                    eid = (int)value;
                else if (index == 1)
                    name = (string)value;
                else if (index == 2)
                    salary = (double)value;
                else
                    Console.WriteLine("Invalid index");
            }
        }

        // string indexer
        public Object this[string index]
        {
            get
            {
                if (index == "eid")
                    return eid;
```

```
----- Indexer -----
----- Details of Employee e1: -----
Id: 401
Name: ram
Salary: 35000.5
Updated salary of e1: 37999.5

----- Details of Employee e2: -----
Id: 401
Name: Gopalnanda
Salary: 37050
Updated name of e2: Gopal
Updated salary of e2: 43050.5
```

```

        else if (index == "name")
            return name;
        else if (index == "salary")
            return salary;
        return null;
    }
    set
    {
        if (index == "eid")
            eid = (int)value;
        else if (index == "name")
            name = (string)value;
        else if (index == "salary")
            salary = (double)value;
        else
            Console.WriteLine("Invalid index");
    }
}
}
class Program
{
    static void Main()
    {
        TestEmployee e1 = new(401, "ram", 35000.5);
        Console.WriteLine($"----- Indexer ----- \n" +
            $"----- Details of Employee e1: ----- \n" +
            $"Id: {e1[0]} \nName: {e1[1]} \nSalary: {e1[2]}");

        // updating employee details
        e1[2] = 37999.5;
        Console.WriteLine($"Updated salary of e1: {e1[2]}");

        TestEmployee e2 = new(401, "Gopalnanda", 37050);
        Console.WriteLine($" \n----- Details of Employee e2: ----- \n" +
            $"Id: {e2["eid"]} \nName: {e2["name"]} \nSalary: {e2["salary"]}");

        // updating employee details
        e2["name"] = "Gopal";
        e2["salary"] = 43050.5;
        Console.WriteLine($"Updated name of e2: {e2["name"]}");
        Console.WriteLine($"Updated salary of e2: {e2["salary"]}");
    }
}

```

```

----- Indexer -----
----- Details of Employee e1: -----
Id: 401
Name: ram
Salary: 35000.5
Updated salary of e1: 37999.5

----- Details of Employee e2: -----
Id: 401
Name: Gopalnanda
Salary: 37050
Updated name of e2: Gopal
Updated salary of e2: 43050.5

```

5. WAP to demonstrate: a) The use of base keyword to access base class fields b) The use of base keyword to call base class methods c) The use of base keyword to call base class constructor

The **base keyword** is used in a derived (child) class to refer to members of its **base (parent) class**.

It allows the child class to:

- Call a constructor of the base class.
- Access or override methods, properties, or fields of the base class that are not hidden or private.

Usage	Purpose
<code>base()</code>	Call a base class constructor from the derived class constructor.
<code>base.Method()</code>	Call a method from the base class (even if overridden).
<code>base.Property</code>	Access a property or field from the base class.

```
using System;

namespace Sushanka_NCC_Lab
{
    class Car
    {
        protected int seats = 4;
        readonly string chasis;
        protected Car() // default constructor
        {
            Console.WriteLine("Inside Base() - Default Constructor");
            chasis = "Unknown";
        }
        protected Car(string chasis) // parameterized constructor
        {
            Console.WriteLine("Inside Base(string chasis) - Parameterized Constructor");
            this.chasis = chasis;
        }
        public void ShowEngine() => Console.WriteLine("Chasis no: " + chasis);
        protected void Status() => Console.WriteLine("I am Parent method."); // works
        only when the body is a single expression
    }
    class Tesla : Car
    {
        readonly new int seats = 2;
        // This constructor will call Car.Car()
        public Tesla() : base() { }
        // This constructor will call Car.Car(string chasis)
        public Tesla(string engine_no) : base(engine_no) { }
    }
}
```



```

    public void Show ()
    {
        Console.WriteLine("Parent field: (seats): " + base.seats);
        Console.WriteLine("Child field: (seats): " + seats);
    }
    new public void Status() // method hiding
    {
        base.Status();
        Console.WriteLine("I'm Child method");
    }

}
class TestBase
{
    static void Main()
    {
        Tesla c1= new ();
        c1.Show();
        c1.Status();
        Console.WriteLine("");

        Tesla c2 = new("C4J37E");
        //c2.Show();
        //c2.Status();
        c2.ShowEngine();
    }
}
}

```

```

Inside Base() - Default Constructor
Parent field: (seats): 4
Child field: (seats): 2
I am Parent method.
I'm Child method

```

```

Inside Base(string chasis) - Parameterized Constructor
Chasis no: C4J37E

```

6. Program to show a) method overriding and method hiding/shadowing in C# b) dynamic polymorphism using method overriding.

Method Overriding

 In a **derived class**, you provide a **new implementation** for a method that was defined as `virtual` or `abstract` in the base class.

 The overridden method replaces the base version when called on a derived object.

 Enables **runtime (dynamic) polymorphism**.

★ Requirements:

- Base method must be marked `virtual`, `abstract`, or `override`.
- Derived method must use `override`.

Method Hiding (Shadowing)

 In a **derived class**, you define a **new method with the same name and signature** as in the base class.

 This hides the base class method rather than overriding it.

 Base method is still there, but hidden when accessed through the derived class reference.

★ Use the `new` keyword in the derived class to explicitly hide the base method.

What is Dynamic Polymorphism?

It means the **method that gets executed is determined at runtime**, depending on the actual type of the object — not the reference type.

This is achieved via **method overriding**.

```

using System;

namespace Sushanka_NCC_Lab
{
    class Base
    {
        public virtual void Override() => Console.WriteLine("Base Method: Overridable.");

        public void Visibility() => Console.WriteLine("Base Method: I'm hiding.");
    }
    class Derived: Base
    {
        public override void Override() => Console.WriteLine("Derived Method: Overridden.");
        public new void Visibility()
        {
            Console.WriteLine("Derived Method: I'm displaying.");
        }
    }
    class Ride_vs_Hide
    {
        static void Main()
        {
            Base b1 = new(); // base classinstance
            b1.Override();

            Derived c1 = new(); // derived class instance
            c1.Override();
            c1.Visibility();

            // instance of derived class; reference to Base class
            Base c2 = new Derived();
            c2.Override(); // calls for Derived class method, because it's overridden
            c2.Visibility(); // calls base class method,
            // the calls go through Animal reference
            // but the actual method executed depends on the actual object (runtime)
        }
    }
}
/*
◆ Why is it called dynamic?
Because the method to execute (Override() from Base, Derived) is chosen at runtime,
based on the object's type, not just the reference type.
*/

```

```

Base Method: Overridable.
Derived Method: Overridden.
Derived Method: I'm displaying.
Derived Method: Overridden.
Base Method: I'm hiding.

```

7. WAP to illustrate the concept of a) Abstract class b) Interface c) Multiple inheritance using interface in C#.

◆ 1 Abstract Class

📖 An abstract class in C#:

- is a class that cannot be instantiated directly.
- can contain:
 - abstract methods (without implementation)
 - non-abstract (regular) methods
 - fields & properties
- derived classes **must** implement the abstract methods.

🔧 Why use?

When you want to provide some common implementation and enforce that derived classes implement specific behaviors.

◆ 2 Interface

📖 An interface:

- is a completely abstract type.
- can only declare:
 - method signatures
 - properties, events, indexers (no fields)
- cannot contain any implementation (until C# 8.0 — where default implementations are allowed in interfaces).
- classes or structs that implement the interface **must implement all its members**.

🔧 Why use?

When you just want to define a *contract* that must be fulfilled, without prescribing how it's done.

◆ 3 Multiple Inheritance using Interface

📖 In C#, a class cannot inherit from more than one class (because C# does not support multiple inheritance of classes).

BUT a class can **implement multiple interfaces**.

🔧 Why use?

This allows you to achieve **multiple inheritance of behavior/contracts**, avoiding the problems of traditional multiple inheritance.

7_a.cs

```
using System;

abstract class Employee
{
    public abstract void Role();
    // non abstract method; has body
    public void Available() => Console.WriteLine("I'm working.");
}

class Manager: Employee
{
    public override void Role() => Console.WriteLine("I'm Manager.");
}

class Developer : Employee
{
    public override void Role() => Console.WriteLine("I'm Developer.");
}

class Abstract_test
{
    static void Main()
    {
        //Employee e1 = new();    error: cannot create instance of the abstract or interface class
        //Employee e2 = new Manager();
        //e2.Role();

        Console.WriteLine("----- Abstract Class -----");
        //Employee e1 = new Manager();
        Employee e1;
        e1 = new Manager();
        e1.Role();
        e1.Available();

        e1 = new Developer();
        e1.Role();
    }
}

/*
 * instance of abstract class cannot be created (but base class can be referenced on child
instance)
 * must override abstract method
 * Non - Abstract method must be declared (have body)
 * Abstract method body can't be declared; can't be private cuz it must be overridden (with
body)
 * Abstract class can contain both abstract method and non-abstract method
 * but class containg abstract method must be an abstract class
 */
```

```
----- Abstract Class -----
I'm Manager.
I'm working.
I'm Developer.
```

7_b_c.cs

```
using System;

namespace Sushanka_NCC_Lab
{
    interface ITelephone
    {
        void Call();
    }
    interface ICamera
    {
        public void Capture();
    }
    class Phone(string name): ITelephone, ICamera // multiple inheritance using interface
    {
        string name= name;
        public void Call()
        {
            Console.WriteLine($" {name} is able to make a call.");
        }
        public void Capture()
        {
            Console.WriteLine($" {name} is able to capture pictures.");
        }
        static void Main()
        {
            Console.WriteLine("----- Multiple Inheritance using Interface -----");
            Phone android= new("Android");
            android.Call();
            android.Capture();
        }
    }
}
```

```
----- Multiple Inheritance using Interface -----
Android is able to make a call.
Android is able to capture pictures.
```

8. WAP program that contains:

- a) Structure (struct)
- b) Enumeration (enum)
- c) Partial class

8_a.cs

```
using System;

namespace Sushanka_NCC_Lab
{
    struct Country
    {
        int code;
        string name;
        Country(int code, string name)
        {
            this.code = code; this.name = name;
        }

        static void Main()
        {
            Country c1 = new(977, "Nepal");
            Console.WriteLine("----- struct Demo ----");
            Console.WriteLine($"Country Details: \n\tName: {c1.name} \n\tCode: {c1.code}");
        }
    }
}
```

```
----- struct Demo ----
Country Details:
    Name: Nepal
    Code:977
```

8_b.cs

```
using System;

namespace Sushanka_NCC_Lab
{
    enum Countries { Nepal, Spain, China, America}
    enum Games { Football, Volleyball, Rugby, Kabbadi}
    class EnumerationDemo
    {
        static void Main()
        {
            Console.WriteLine("----- Enumeration -----");
            Console.WriteLine("Integral value of {0} is {1}", Countries.Nepal,
(int)Countries.Nepal);

            foreach(Games game in Enum.GetValues(typeof(Games)))
            {
                Console.WriteLine(game + "=" + (int)game);
            }
        }
    }
}
```

```
----- Enumeration -----
Integral value of Nepal is 0
Football=0
Volleyball=1
Rugby=2
Kabbadi=3
```

8_c_1.cs

```
using System;

namespace Sushanka_NCC_Lab
{
    partial class PartialClass
    {
        public void SayNamaste() => Console.WriteLine("Namaste from Partial Class part
1");
    }
}
```


8_c_2.cs

```
using System;

namespace Sushanka_NCC_Lab
{
    // partial class part 2
    partial class PartialClass
    {
        public void SayHello() => Console.WriteLine("Hello from Partial Class part 2");
    }
    class PartialProgram
    {
        static void Main()
        {
            Console.WriteLine("***** Partial Class *****");
            PartialClass c1 = new ();
            c1.SayNamaste();
            c1.SayHello();
        }
    }
}
```

```
***** Partial Class *****
Namaste from Partial Class part 1
Hello from Partial Class part 2
```

9. WAP to illustrate the concept of: a) Delegate b) Multicast delegate c) Func Delegate d) Action Delegate e) Anonymous Method f) Event in C#.

a) Delegate

```
using System;

namespace Sushanka_NCC_Lab
{
    delegate void Health_Check();
    class BMI
    {
        /// <summary>
        /// BMI = weight(kg) / height(m²).
        /// </summary>
        /// <param name="name">Patient Name</param>
        /// <param name="weight">Patient weight in kg</param>
        /// <param name="height">Patient height in m</param>
        void BMI_Calculate(string name, float weight, float height) // BMI = weight (kg) /
        height (m²).
        {
            float bmi_index = weight / (height * height);
            Console.WriteLine($"Patient Details: \n\tName: {name} \n\tBMI_Index =
{bmi_index:F2} ");
            string msg = "";
            if (bmi_index < 18.5)
                msg = "Underweight";
            else if (bmi_index < 25)
                msg = "Healthy Weight";
            else if (bmi_index < 30)
                msg = "Overweight";
            else
                msg = "Obese";
            Console.WriteLine($"Status: {msg}");
        }
        static void Main()
        {
            Console.WriteLine($"----- Delegate -----");
            BMI p1 = new();
            p1.BMI_Calculate(name: "Harkaraj", weight:45.85f, height:1.4f); // weight(kg) ---
> height(m)
        }
    }
}
```

```
----- Delegate -----
Patient Details:
    Name: Harkaraj
    BMI_Index = 23.39
    Status: Healthy Weight
```

b) Multicast delegate

```
using System;

namespace Sushanka_NCC_Lab
{
    // delegate multicasting
    public delegate void DelegateDemo(string name, int age);
    class Citizen
    {
        void Voters_Check(string name, int age)
        {
            if (age >= 18)
                Console.WriteLine($"Congrats {name}!!! Eligible for Voting");
            else
                Console.WriteLine($"Sorry {name}!!!. Not Eligible for Voting");
        }
        void Elders_Check(string name, int age)
        {
            if (age >= 70)
                Console.WriteLine($"Congrats {name}!!! Eligible for Social Security Allowance");
            else
                Console.WriteLine($"Sorry {name}!!!. Not Eligible for Social Security Allowance");
        }
        static void Main()
        {
            /*
            Citizen c1 = new();
            DelegateDemo dd = c1.Voters_Check;
            dd.Invoke("harry", 42);
            dd = c1.Elders_Check;
            dd.Invoke("haru", 43);
            */

            // instead use delegate multicast
            Console.WriteLine("----- Delegate Multicast -----");
            Citizen c1 = new();
            DelegateDemo dd1 = c1.Voters_Check;
            DelegateDemo dd2 = c1.Elders_Check;
            DelegateDemo dd = dd1 + dd2;
            dd.Invoke("Godattho Prasad", 67);
        }
    }
}
```

```
----- Delegate Multicast -----
Congrats Godattho Prasad!!! Eligible for Voting
Sorry Godattho Prasad!!!. Not Eligible for Social Security Allowance
```

c) Func Delegate d) Action Delegate e) Anonymous Method

```
using System;

namespace Sushanka_NCC_Lab
{
    class DelegateExample
    {
        static void Main()
        {
            Console.WriteLine("----- Func Delegate -----");
            // func delegate takes 2 int parameters and return int
            Func<int, int, int> add = (x, y) => x + y;
            int sum = add(97, 3);
            Console.WriteLine($"Sum: {sum}");

            Console.WriteLine("----- Action Delegate -----");
            // action delegate takes 1 string parameter and return void
            Action<string> greet = (name) => .WriteLine($"Namaskar,
{name}");
            greet("Topendra");

            Console.WriteLine("----- Anonymous Delegate -----");
            // Anonymous Method: Assigned to Func delegate
            Func<int, int, int> multiply = delegate (int x, int y)
            {
                return x * y;
            };
            int product = multiply(97, 3);
            Console.WriteLine($"Product: {product}");
        }
    }
}
```

```
----- Func Delegate -----
Sum: 100
----- Action Delegate -----
Namaskar, Topendra
----- Anonymous Delegate -----
Product: 291
```

f) Event in C#.

```
using System;

namespace Sushanka_NCC_Lab
{
    // Define a delegate for the event
    delegate void Notify(string message);        // internal inside
    namespace
    {
        class Publisher    // equivalent to internal Publisher{.....} by
        default internal Top-level (inside namespace)
        {
            // Declare the event using the delegate
            public event Notify OnNotify;
            public void DoSomething()
            {
                Console.WriteLine("Publisher: Doing sth important...");

                // Raise the event
                OnNotify?.Invoke("Task Completed!");
            }
        }
        class Subscriber_A
        {
            public void HandleNotifcation(string msg) =>
            Console.WriteLine($"Subscriber A received: {msg}");
        }
        class Subscriber_B
        {
            public void HandleNotifcation(string msg) =>
            Console.WriteLine($"Subscriber B received: {msg}");
        }
        class EventHandlering // Main Program
        {
            static void Main()
            {
                Console.WriteLine("----- Events In C# -----");
                Publisher pub= new();
                // create subscribers
                Subscriber_A sub_a = new();
                Subscriber_B sub_b = new();

                // register(subscribe) to the event
                pub.OnNotify += sub_a.HandleNotifcation;
                pub.OnNotify += sub_b.HandleNotifcation;

                pub.DoSomething();
            }
        }
    }
}
```

```
----- Events In C# -----
Publisher: Doing sth important...
Subscriber A received: Task Completed!
Subscriber B received: Task Completed!
```

10. WAP which use any
a) Non generic collection

```
using System;
using System.Collections;

namespace Sushanka_NCC_Lab
{
    class NonGeneric
    {
        static void Main()
        {
            ArrayList a1 = new();
            a1.Add("Non-Generic");
            a1.Add("Generic");
            a1.Add("Concurrent");
            Console.WriteLine($"Non-Generic Collection\n" +
                              $"\nCounts: {a1.Count}" +
                              $"\nCapacity: {a1.Capacity}");

            foreach(string i in a1)
            {
                Console.Write($"{i}, ");
            }
        }
    }
}
```

Non-Generic Collection

Counts: 5

Capacity: 8

Non-Generic, Generic, Concurrent, 10, 40.54,

b) Generic Collection

```
using System;
using System.Collections.Generic;

namespace Sushanka_NCC_Lab
{
    class Generic
    {
        static void Sum<T, F>(T a, F b)
        {
            dynamic x = a, y = b;
            Console.WriteLine($"{a} + {b} = {x + y}");
        }
        static void Mul<T, F>(T a, F b)
        {
            dynamic x = a, y = b;
            Console.WriteLine($"{a} * {b} = {x * y}");
        }
        //static void Add(int a, int b) => Console.WriteLine($"{a} +
        {b} = {a+b}");
        //static void Mul(int a, int b) => Console.WriteLine($"{a} *
        {b} = {a * b}");
        //static void Mul(float a, float b) =>
        Console.WriteLine($"{a} * {b} = {a * b}");
        static void Main()
        {
            Console.WriteLine("----- Generic Collection -----");
            Console.WriteLine("**** Sum ****");
            Sum(1, 2);
            Sum(1.53f, 2.0f);
            Sum<int, float>(1, 2.4f);    // By default compiler
            understand it or Sum(1, 2.4f)
            Console.WriteLine("**** Product ****");
            Mul(1, 2);
            Mul(1.53f, 2.0f);
            Mul(1, 2.4f);
        }
    }
}
```

```
----- Generic Collection -----
**** Sum ****
1 + 2 = 3
1.53 + 2 = 3.53
1 + 2.4 = 3.4
**** Product ****
1 * 2 = 2
1.53 * 2 = 3.06
1 * 2.4 = 2.4
```

11. Program to demonstrate the use of Generic Class with Generic field and method.

```
//11. Program to demonstrate the use of Generic Class with Generic
field and method.
using System;
using System.Collections.Generic;

namespace Sushanka_NCC_Lab
{
    // generic class
    class GenericDemo<T>
    {
        // define a generic field
        T data;
        // generic constructor
        public GenericDemo(T value) => data = value;
        public void DisplayData() => Console.WriteLine($"Data:
{data}");

        // generic method
        public void ShowType<U>(U parameter)
        {
            Console.WriteLine($"Type of T: {typeof(T)} data:
{data}");
            Console.WriteLine($"Type of U: {typeof(U)} data:
{parameter}");
        }
    }

    class GenericProgram
    {
        static void Main()
        {
            Console.WriteLine("Generic Fields and Methods");
            GenericDemo<int> obj1 = new(23);
            obj1.DisplayData();
            obj1.ShowType<double>(3.14);

            GenericDemo<string> obj3 = new("Generic");
            obj3.DisplayData();
            obj3.ShowType<char>('G');
        }
    }
}
```

```
Generic Fields and Methods
Data: 23
Type of T: System.Int32 data: 23
Type of U: System.Double data: 3.14
Data: Generic
Type of T: System.String data: Generic
Type of U: System.Char data: G
```


12.WAP to take input from keyboard and write them to a file

```
// WAP to take input from keyboard and write them to a file

using System;
using System.IO;

namespace Sushanka_NCC_Lab
{
    class FileDemo
    {
        static void Main()
        {
            Console.WriteLine("----- File Handelling -----");
            Console.WriteLine("\nContents to write: ");
            string? data = Console.ReadLine(); // ? refers to nullable
            string myfilepath =
@"C:\Users\me\source\repos\Sushanka_NCC_Lab\Sushanka_NCC_Lab\demo.txt";

            File.AppendAllText(myfilepath, data);

            Console.WriteLine("\nContents of file:");
            Console.WriteLine(File.ReadAllText(myfilepath));
        }
    }
}

/*
    FileStream f = new(myfilepath, FileMode.Append, FileAccess.Write);
    for (int i = 65; i <= 90; i++)
        f.WriteByte((byte)i);

    foreach(char c in data)
        f.WriteByte((byte)c);
    f.Close();

    f = new(myfilepath, FileMode.Open, FileAccess.Read);
    int j = 0;
    while ((j = f.ReadByte()) != -1)
        Console.Write((char)j);
*/
```

```
----- File Handelling -----
Contents to write:
Lab of Net Centric Computing

Contents of file:
Welcome Back!Lab of Net Centric Computing
```

13.WAP to demonstrate the concept of LINQ

```
// Concept of Language Integrated Query (LINQ)

using System;
using System.Collections;

namespace Sushanka_NCC_Lab
{
    class LINQ_Demo
    {
        static void Main()
        {
            Console.WriteLine("----- Language Integrated Query (LINQ) -----");

            //IList<string> items = new List<string>() { "working",
            "great", "yellow", "wood", ".net", "centric"};
            IList<string> items = ["working", "great", "yellow", "wood",
            ".net", "centric", "server", "backend", "chunk"];

            //int[] items = { 1, 2, 3, 4, 5 };
            var result = from res in items
                        where (res.Contains('e') || res.Contains('i'))
                        orderby res descending
                        select res;

            Console.WriteLine("Query Result:");
            foreach (string item in result)
                Console.Write(item + ", ");
            Console.WriteLine();
        }
    }
}

/*
Enumerable ->>> IEnumerable <T>
Queryable -->>> IQueryable <T>
*/

//int[] items = { 1, 2, 3, 4, 5 };

//var result= from r in items select r;
//Console.WriteLine(result);
//foreach (var item in items)
//Console.WriteLine(item);
```

```
----- Language Integrated Query (LINQ) -----
Query Result:
yellow, working, server, great, centric, backend, .net,
```

14.WAP to a) demonstrate Lamda Expressions in C#. b) LINQ with Lamda Expression in C#

```
using System;

namespace Sushanka_NCC_Lab
{
    class LambdaExp
    {
        // expression lambda
        //Func<int, int, string> Sum= (x, y) => $"Sum of {x} and {y} is {x+y}";
        Action<int, int> Sum = (x, y) => Console.WriteLine($"Sum of {x} and {y} is {x + y}"); // no return type(action)

        // statement lambda
        Func<string, string> Get_Ascii= (x) => // can have dynamic as return type for objects, dictionary, list
        {
            string ascii_kvp = "";
            //List<dynamic> Ascii_list = [];
            foreach (char c in x)
                ascii_kvp += $"{c}:{(int)c}, ";
            return ascii_kvp;
        };

        static void Main()
        {
            Console.WriteLine("----- Lambda Expression -----");
            Console.WriteLine("Expression:");
            LambdaExp l1 = new ();
            l1.Sum(3, 4);

            Console.WriteLine("\nStatement Lambda:");
            string msg = "ASCII", msg_ascii =l1.Get_Ascii(msg);
            Console.WriteLine($"Char ASCII Pair of {msg}:\n{msg_ascii}");
        }
    }

    /*
    Lambda Expression:
        * Expression Lambda
        * Statement Lambda
        *
    if no return type use Action<>

    any lambda expression can be converted to delegate type
    if lambda exp returns value it can be converted to one of Func delegate
    else: => to Action delegate

    Dictionary<char, int> Ascii_dict = new ()
    {
        { 'A', 65 },
    };

    or

```

```

Dictionary<char, int> Ascii_dict = new();
foreach (char c in x)
    Ascii_dict.Add(c, (int)c);
*/
/* custom delegates + lambda expressions
*delegate int SquareDelegate(int num);
    delegate void HelloDelegate(string name);
    class DelegateExample
    {
        static void Main()
        {
            HelloDelegate hd = (name) =>
            {
                string msg = "Hello" + name + ", Good Morning";
                Console.WriteLine(msg);
            };
            SquareDelegate sd = (num) => num * num;

            hd("jack");
            Console.WriteLine(sd(7));
        }
    }
*/

```

```

----- Lambda Expression -----
Expression:
Sum of 3 and 4 is 7

Statement Lambda:
Char ASCII Pair of ASCII:
A:65, S:83, C:67, I:73, I:73,

```

15.WAP a) to use built-in attributes in C#. b) to create and use custom attribute in C#

⚡ Exception Handling

Exception handling is a mechanism to handle runtime errors in a program gracefully, without crashing it. In C#, exceptions are objects derived from `System.Exception`.

Keyword	Purpose
try	Contains code that may throw an exception
catch	Handles the exception
throw	Throws an exception
finally	Executes regardless of exception, for cleanup
custom exception	User-defined exception for specific scenarios

```

using System;
namespace Sushanka_NCC_Lab
{
    class ComplexNumberException: Exception
    {
        public ComplexNumberException(string msg) : base(msg) { }
    }
    class Not_Perfect_Square_Root(string msg) : Exception(msg);
    class ExceptionHandelling
    {
        static void SquareRoot(int x)
        {
            double sqrt = Math.Sqrt(x);
            if (x < 0)
                // throw custom exception
                throw new ComplexNumberException("Complex number
exception. Must be positive integer.");
            if (sqrt != (int)sqrt) // throw if not perfect sq root
                throw new Not_Perfect_Square_Root("Perfect Square
exception. Not a perfect square.");
            Console.WriteLine($"Sqrt of {x} is :{Math.Sqrt(x)}");
        }
        static void Main()
        {
            Console.WriteLine("----- Exception Handeling -----");
            try
            {
                SquareRoot(-5);
                //SquareRoot(25);
            }
            catch (ComplexNumberException e) {
Console.WriteLine(e.Message); }
            catch (Not_Perfect_Square_Root e) // custom exception
            {
                Console.WriteLine(e.Message);
            }
            catch (Exception e) // General exception
            {
                Console.WriteLine(e.Message);
            }

            Finally { Console.WriteLine("Finally block executed."); }
            Console.WriteLine("Program continues after handling custom
exception.");
        }
    }
}

```

```

----- Exception Handeling -----
Complex number exception. Must be positive integer.
Finally block executed.
Program continues after handling custom exception.

```

16. WAP a) to use built-in attributes in C#. b) to create and use custom attribute in C#

16_a.cs

```
using System;
using System.Text.Json;

namespace Sushanka_NCC_Lab
{
    [Serializable]
    class Person_Demo {
        //public required string Name { get; set; }
        public required string Name { get; set; }
        public required string Nationality { get; set; }
        public required string Gender { get; set; }
    }

    class Built_In_Attributes_Demo
    {
        [Obsolete("Use NewMethod instead")]
        static void OldMethod() => Console.WriteLine("This is old
method.");
        static void NewMethod() => Console.WriteLine("This is new
method.");
        static void Main()
        {
            Console.WriteLine("----- Built-in Attributes-----");
            Console.WriteLine("[Serializable]");

            Person_Demo p = new() { Name = "Sushanka", Gender="Male",
Nationality="Nepali" };
            //Person_Demo p = new() { };

            // Serialize to JSON
            string json = JsonSerializer.Serialize(p);
            string myfilepath =
@"C:\Users\me\source\repos\Sushanka_NCC_Lab\Sushanka_NCC_Lab\person_demo.
dat";

            File.WriteAllText(myfilepath, json);
            Console.WriteLine("Serialized JSON\n" + json);

            // Deserialize from JSON
            string readJson = File.ReadAllText(myfilepath);
            Person_Demo? p2 =
JsonSerializer.Deserialize<Person_Demo>(readJson);

            Console.WriteLine($"Deserialize: \nName= {p2?.Name}
Nationality= {p2?.Nationality}, Gender= {p2?.Gender}");

            Console.WriteLine("\n[Obsolete]");
            OldMethod();    // show a compiler warnig because of
[Obsolete]
            NewMethod();
            Console.WriteLine("Built-in attributes demo completed.");
        }
    }
}
```

Solution ▾ 0 Errors 1 Warning 0 Messages Build + IntelliSense ▾

Code	Description	Project
CS0618	'Built_In_Attributes_Demo.OldMethod()' is obsolete: 'Use NewMethod instead'	Sushanka_NCC_Lab

```
----- Built-in Attributes-----  
[Serializable]  
Serialized JSON  
{ "Name": "Sushanka", "Nationality": "Nepali", "Gender": "Male" }  
Deserialize:  
Name= Sushanka Nationality= Nepali, Gender= Male  
  
[Obsolete]  
This is old method.  
This is new method.  
Built-in attributes demo completed.
```

◆ Attribute

- ✓ Attributes are metadata attached to code elements (like classes, methods, properties, etc.) that provide additional information to the compiler, tools, or runtime.
- ✓ Attributes are enclosed in square brackets `[]`.
- ✓ Examples of **built-in attributes**:
 - `[Obsolete]` — marks code as obsolete
 - `[Serializable]` — marks a class as serializable
 - `[DllImport]` — used for interop with native libraries

📄 Why use attributes?

- To give declarative information about your code.
- To control runtime behavior or inform tools about code features.

```

using System;

namespace Sushanka_NCC_Lab
{
    // define a custom attribute
    [AttributeUsage(AttributeTargets.Method | AttributeTargets.Class)]
    class Developer_Info_Attribute : Attribute
    {
        public string Developer { get; }
        public string Date { get; }

        public Developer_Info_Attribute(string developer, string date)
        {
            Developer = developer;
            Date = date;
        }
    }

    class Customer_Attribute_Demo
    {
        [Developer_Info("Sushanka", "2082-4-2")]
        public static void Some_Method()
        {
            Console.WriteLine("Some_Method executed.");
        }

        static void Main()
        {
            Console.WriteLine("----- Custom Attributes -----");
            Some_Method();
            // read the custom attribute at runtime using reflection
            var methodInfo =
typeof(Customer_Attribute_Demo).GetMethod("Some_Method");
            var attributes =
methodInfo.GetCustomAttributes(typeof(Developer_Info_Attribute), false);

            foreach(Developer_Info_Attribute attr in attributes)
                Console.WriteLine($"Developer: {attr.Developer}, Date:
{attr.Date}");
        }
    }
}

```

```

----- Custom Attributes -----
Some_Method executed.
Developer: Sushanka, Date: 2082-4-2

```


17.WAP to demonstrate asynchronous programming in C# using async and await keywords.

🌟 Explanation:

- ✅ `async` — marks a method as asynchronous.
- ✅ `await` — tells the compiler to wait for the result of an async operation without blocking the main thread.
- ✅ `Task` — represents the ongoing operation.

Here:

- `Main` is marked `async Task` to allow `await` inside it.
- `PerformTaskAsync()` does some asynchronous work (`Task.Delay` simulates a delay).

```
using System;
using System.Threading.Tasks;

namespace Sushanka_NCC_Lab
{
    class AsyncClass
    {
        // Async method
        static async Task LongRunningTask()
        {
            Console.WriteLine("Task Started...");
            await Task.Delay(3000); // simulate a delay of 3 seconds
            Console.WriteLine("Task Completed after 3 seconds.");
        }

        static void Main()
        {
            Console.WriteLine("----- Asynchronous Programming -----");
            Console.WriteLine("Main method started.");

            // call the async method
            Task task = LongRunningTask();

            Console.WriteLine("Main method continues while task
runs..."); ;

            // wait for task to complete
            task.Wait();

            Console.WriteLine("Main Method ends.");
        }
    }
}
```

```
----- Asynchronous Programming -----
Main method started.
Task Started....
Main method continues while task runs...
Task Completed after 3 seconds.
Main Method ends.
```